

# **AI 雙認證全攻略：AI 應用規劃師 & 工程素養認證**

AI 仁波切

2025-11-20

# 目錄

|       |                                                       |    |
|-------|-------------------------------------------------------|----|
| 1     | 書名：AI 雙認證全攻略：AI 應用規劃師 & 工程素養認證                        | 4  |
| 2     | 第一篇：人工智慧與機器學習基礎 (AI & Machine Learning Fundamentals)  | 5  |
| 2.1   | 第一章：人工智慧概論 (Introduction to AI)                       | 5  |
| 2.1.1 | 什麼是人工智慧？                                              | 5  |
| 2.1.2 | AI 的三種型態                                              | 5  |
| 2.2   | 第二章：機器學習基礎 (Machine Learning Basics)                  | 6  |
| 2.2.1 | 教電腦「學習」而不是「聽令」                                        | 6  |
| 2.2.2 | 常見的機器學習模型 (你的 AI 工具箱)                                 | 6  |
| 2.3   | 第三章：深度學習與神經網路 (Deep Learning & Neural Networks)       | 9  |
| 2.3.1 | 模仿大腦的運作                                               | 9  |
| 2.3.2 | 深度學習的三大巨頭                                             | 10 |
| 2.4   | 第四章：建模與調校 (Modeling & Tuning)                         | 10 |
| 2.4.1 | 打造完美的 AI 模型                                           | 10 |
| 3     | 第二篇：AI 技術應用與生成式 AI (AI Applications & Generative AI)  | 12 |
| 3.1   | 第五章：鑑別式與生成式 AI (Discriminative vs Generative AI)      | 12 |
| 3.1.1 | 兩大 AI 陣營的對決與合作                                        | 12 |
| 3.1.2 | 當判官遇見藝術家：整合應用                                         | 13 |
| 3.2   | 第六章：生成式 AI 應用與工具 (Generative AI Tools & Applications) | 13 |
| 3.2.1 | No Code / Low Code：AI 民主化運動                           | 13 |
| 3.2.2 | 駕馭 AI 的關鍵技能：Prompt Engineering (提示工程)                 | 13 |
| 3.2.3 | 生成式 AI 的導入與風險                                         | 14 |
| 3.3   | 第七章：進階 AI 技術與部署 (Advanced AI Tech & Deployment)       | 14 |
| 3.3.1 | 深入 AI 的技術核心                                           | 14 |
| 3.3.2 | NLP 與 CV：AI 的聽說讀寫                                     | 15 |
| 3.3.3 | AI 系統部署：從實驗室到真實世界                                     | 15 |
| 4     | 第三篇：資料科學與大數據 (Data Science & Big Data)                | 16 |
| 4.1   | 第八章：資料處理基礎 (Data Processing Basics)                   | 16 |
| 4.1.1 | 資料：AI 的糧食                                             | 16 |
| 4.2   | 第九章：大數據技術與應用 (Big Data Technologies)                  | 17 |
| 4.2.1 | 當資料大到無法想像：大數據 (Big Data)                              | 17 |
| 4.2.2 | 大數據的統計學基礎                                             | 17 |
| 4.2.3 | 駕馭大數據的工具                                              | 17 |
| 4.2.4 | 大數據與 AI 的強強聯手                                         | 18 |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>5</b> | <b>第四篇：AI 倫理 治理與風險 (AI Ethics, Governance &amp; Risks)</b> | <b>19</b> |
| 5.1      | 第十章：AI 治理與風險 (AI Governance & Risks) . . . . .             | 19        |
| 5.1.1    | AI 的潛在威脅 . . . . .                                         | 19        |
| 5.2      | 第十一章：AI 倫理 (AI Ethics) . . . . .                           | 20        |
| 5.2.1    | AI 應該遵守的道德規範 . . . . .                                     | 20        |
| 5.2.2    | 案例分析：AI 倫理的兩難 . . . . .                                    | 21        |
| <b>6</b> | <b>第五篇：AI 程式語言 (AI Programming)</b>                        | <b>22</b> |
| 6.1      | 第十二章：Python 基礎 (Python Basics) . . . . .                   | 22        |
| 6.1.1    | 為什麼是 Python ? . . . . .                                    | 22        |
| 6.1.2    | Python 語法速成 . . . . .                                      | 22        |

# 1 書名：AI 雙認證全攻略：AI 應用規劃師 & 工程素養認證

**適用對象：**準備經濟部 iPAS AI 應用規劃師(初/中級)及資策會人工智慧工程素養認證之考生。

本攻略旨在協助考生掌握 AI 應用規劃師與工程素養認證之核心知識，內容涵蓋人工智慧基礎、機器學習、深度學習、生成式 AI、大數據分析、AI 倫理與法規，以及 Python 程式語言基礎。

請點選左側目錄開始閱讀各章節。

## 2 第一篇：人工智慧與機器學習基礎 (AI & Machine Learning Fundamentals)

歡迎來到人工智慧 (AI) 的世界！這本書將帶領你從零開始，深入淺出地理解 AI 的核心概念、運作原理以及它如何改變我們的生活。我們不需要複雜的數學公式，而是用直觀的語言和生動的例子，讓你輕鬆掌握這些看似高深的技術。

### 2.1 第一章：人工智慧概論 (Introduction to AI)

#### 2.1.1 什麼是人工智慧？

簡單來說，人工智慧 (Artificial Intelligence, AI) 就是讓電腦擁有像人類一樣的「智慧」。這包括學習、推理、解決問題、理解語言，甚至感知環境的能力。想像一下，傳統的電腦程式就像是一個只會聽命行事的工廠作業員，你必須給它明確的指令 (程式碼)，它才會動作。而 AI 則像是一個聰明的學徒，它可以透過觀察數據和經驗，自己學會如何完成任務。

#### 2.1.2 AI 的三種型態

我們可以將 AI 依據其能力的強弱，分為三個等級：

**Figure Prompt:** Generate a flat illustration comparing three types of AI. 1. Weak AI: A smartphone showing Siri/Assistant icon. 2. Strong AI: A robot shaking hands with a human, looking equal. 3. Super AI: A glowing, abstract digital brain hovering above a city, symbolizing superior intelligence. Style: Modern, clean, vector art.

1. **弱人工智慧 (Artificial Narrow Intelligence, ANI)**：這是我們目前生活中最常見的 AI。它們是「專才」，非常擅長做某一件特定的事情，甚至做得比人類更好，但在其他方面則一竅不通。
  - 例子：打敗圍棋冠軍的 AlphaGo、手機裡的 Siri 語音助理、Netflix 的影片推薦系統。AlphaGo 雖然下圍棋無敵，但它不會開車，也不會聊天。
2. **強人工智慧 (Artificial General Intelligence, AGI)**：這是科學家們努力的目標，也是科幻電影中常見的 AI。它們是「通才」，具備像人類一樣的通用智慧，可以學習任何人類能學會的技能，具備常識、情感和自我意識。

- **例子**：電影《雲端情人 (Her)》中的薩曼莎 或是《鋼鐵人》中的賈維斯。目前我們尚未真正達成 AGI。

3. **超人工智慧 (Artificial Super Intelligence, ASI)**：這是理論上 AI 發展的終極階段。當 AI 的智慧在所有領域(包括創造力、社交技巧、問題解決能力)都遠遠超越最聰明的人類時，我們就稱之為超人工智慧。這是一個充滿未知與想像的領域。

---

## 2.2 第二章：機器學習基礎 (Machine Learning Basics)

### 2.2.1 教電腦「學習」而不是「聽令」

機器學習 (Machine Learning, ML) 是 AI 的一個子領域，也是目前 AI 發展的核心動力。

- **傳統程式設計**：我們輸入「規則」和「數據」，電腦給出「答案」。例如：我們告訴電腦「如果下雨，就帶傘」，電腦看到「下雨」的數據，就會輸出「帶傘」。
- **機器學習**：我們輸入「數據」和「答案」，電腦自己找出「規則」。例如：我們給電腦看一萬張貓的照片(數據)並告訴它這些是貓(答案)，電腦就會自己分析出貓的特徵(有尖耳朵、鬍鬚...)，最後學會如何辨識貓。

**Figure Prompt:** Generate a diagram contrasting “Traditional Programming” vs “Machine Learning”. Top flow: Rules + Data -> [Traditional Programming] -> Answers. Bottom flow: Data + Answers -> [Machine Learning] -> Rules. Use icons for Data (document), Rules (scroll), and Answers (lightbulb).

### 2.2.2 常見的機器學習模型 (你的 AI 工具箱)

機器學習有很多種方法(模型)，就像工具箱裡有不同的工具，適用於不同的問題。

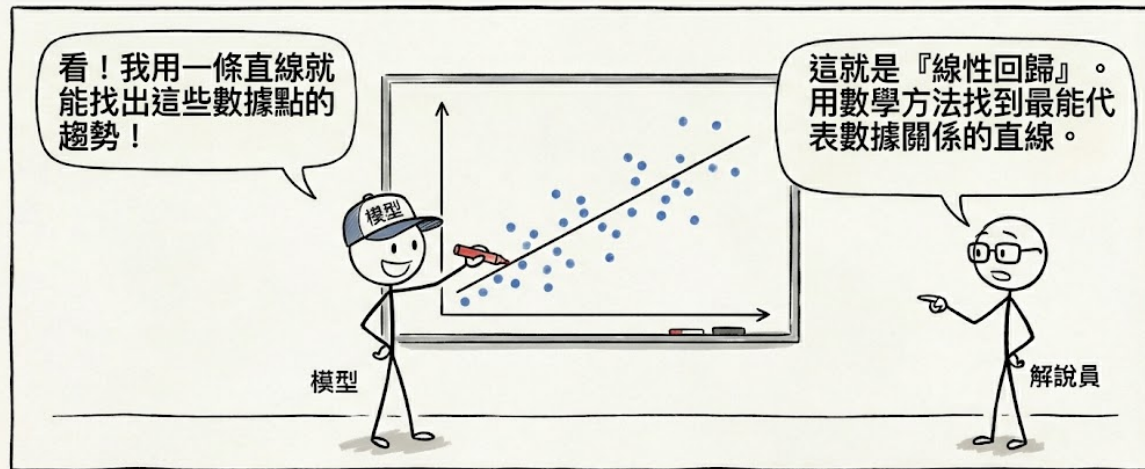
#### 1. 線性回歸 (Linear Regression)：

- **概念**：想像你在畫一張圖，橫軸是房子的大小，縱軸是房價。你會發現點點大致分佈在一條線附近。線性回歸就是試著畫出這條「最佳直線」，用來預測未來的趨勢。
- **數學原理 (Friendly Math)**：其實這條線的公式非常簡單，就是國中學過的：

$$y = wx + b$$

- **y (預測值)**：我們想知道的答案(例如：房價)。
- **x (輸入值)**：我們已知的資訊(例如：房子坪數)。
- **w (權重 Weight)**：這代表影響力的大小。例如：每多一坪，房價會增加多少？這個「多少」就是  $w$  (斜率)。

## 線性回歸 (Linear Regression)



線性回歸：一種用於建模變數之間關係的統計方法，目標是找到一條能夠最小化預測誤差的直線。

圖 2.1: 線性回歸

- $b$  (偏差 Bias) : 這代表起跑點，就算坪數是 0，房子也不會是免費的 (可能有地段的基本價值)。這個基本值就是  $b$  (截距)。

AI 的任務 就是透過看大量的數據，自動算出最準確的  $w$  和  $b$  是多少。

- 應用：預測房價、股票走勢、銷售量。
- 如何知道準不準？(RSE)：我們怎麼知道這條線畫得好不好？這就要看 RSE (殘差標準誤, Residual Standard Error)。
  - 概念：想像這條線是「標準答案」，但真實的房價 (紅點) 通常不會剛好落在線上，而是會散落在線的上下。RSE 就是在計算這些點平均偏離這條線多遠。
  - 解讀：RSE 越小越好。
    - \* 如果  $RSE = 0$ ，代表所有點都在線上，預測完美 (但現實中幾乎不可能)。
    - \* 如果 RSE 很大，代表點點分佈得很散，這條線的預測能力很差。
  - 公式 (Friendly Math)：

$$RSE = \sqrt{\frac{1}{n-2} \sum (y_i - \hat{y}_i)^2}$$

別被嚇到了！簡單來說就是：把每個點的誤差  $(y - \hat{y})$  平方加起來，除以點的數量，再開根號，就像是算所有誤差的「平均值」。

### 2. 邏輯回歸 (Logistic Regression)：

- 概念：雖然名字有「回歸」，但它其實是用來做「分類」的，它像是一個開關，判斷事情是「是」或「否」(0 或 1)。

- **數學原理 (Friendly Math)** : 它其實就是把線性回歸的結果「丟進一個 S 型的函數 (Sigmoid 函數) 裡面 :

$$P(y = 1) = \frac{1}{1 + e^{-(wx+b)}}$$

- **Sigmoid 函數** : 這個函數很神奇 , 不管你輸入多大或多小的數字 , 它吐出來的結果永遠在 0 到 1 之間。
- **機率** : 這個 0 到 1 的數字 , 就代表「機率」, 例如算出 0.8 , 就代表有 80% 的機率是垃圾郵件。
- **決策** : 通常我們設 0.5 為門檻 , 大於 0.5 就是「是」, 小於 0.5 就是「否」。
- **應用** : 判斷電子郵件是不是垃圾郵件 , 這筆交易是不是詐騙。

### 3. 決策樹 (Decision Tree) :

- **概念** : 這就像是在玩「20 個問題」遊戲 , 透過一連串的「是/否」問題 , 將數據層層分類 , 直到得出結論。
- **數學原理 (Friendly Math)** : 決策樹怎麼知道要先問哪個問題 ? 它會算一個叫做「**亂度 (Entropy)**」或「**基尼係數 (Gini Impurity)**」的東西。
  - **亂度** : 想像一個房間裡有紅球和藍球 , 如果紅藍各半 , 亂度最大 (最難猜) ; 如果全是紅球 , 亂度為 0 (最好猜)。
  - **資訊獲利 (Information Gain)** : AI 會試著問一個問題 (例如 : 球是大顆的嗎 ?) , 如果問完之後 , 分出來的兩堆球「亂度」變小了 , 代表這個問題問得好 ! AI 就是一直在找能讓亂度下降最快的問題。
- **結構** : 樹根是第一個問題 , 樹枝是選項 , 樹葉是最終的分類結果。
- **應用** : 銀行審核貸款 (年收入 > 100 萬 ? 是 -> 信用良好 ? 是 -> 核准)。

**Figure Prompt:** Generate a visualization of a Decision Tree. The root node asks “Is it raining?”. Left branch “Yes” -> “Is it windy?” -> “Don’t go out”. Right branch “No” -> “Go out”. Style: Clean, flowchart style with icons.

### 4. 隨機森林 (Random Forest) :

- **概念** : 俗話說「三個臭皮匠 , 勝過一個諸葛亮」, 隨機森林就是種植很多棵「決策樹」, 然後讓它們投票 , 如果大部分的樹都說是「A 類」, 那結果就是 A , 這通常比單一決策樹更準確。
- **數學原理 (Friendly Math)** :
  - **集成學習 (Ensemble Learning)** : 這背後的數學原理叫做「大數法則」, 假設每一棵樹的準確率只有 60% (比亂猜好一點) , 但如果我們有 100 棵樹一起投票 , 犯錯的機率就會大幅下降。
  - **隨機性** : 為什麼叫「隨機」 ? 因為每棵樹看到的數據都不太一樣 (隨機抽樣) , 這樣它們才會有不同的觀點 , 投票起來才客觀。

### 5. 支援向量機 (SVM) :



- **概念**：想像桌上有紅球和藍球混在一起。SVM 的任務就是拿一根棍子（在二維平面是線，三維空間是面）試著把紅球和藍球分得越開越好，這根棍子就是「最佳超平面」。
- **數學原理 (Friendly Math)**：SVM 不只要分開，還要找「**最寬的馬路 (Margin)**」。
  - **Margin**：這條線到最近的紅球和最近的藍球的距離。SVM 想要讓這個距離最大化。
  - **公式**：目標是 Maximize  $\frac{2}{\|w\|}$ 。簡單說，就是讓路越寬越好，這樣以後有新的球丟進來，才不會容易判斷錯誤。

#### 6. K-近鄰演算法 (KNN)：

- **概念**：物以類聚。當來了一個新數據，KNN 會看它最近的 K 個鄰居是誰。如果鄰居大多是紅球，那它大概也是紅球。
- **數學原理 (Friendly Math)**：怎麼算「最近」？就是用我們國中學過的「**距離公式 (Euclidean Distance)**」：

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

- AI 會算出新點跟所有舊點的距離，然後挑出最近的 K 個（例如 K=3）。
- 如果這 3 個鄰居是「紅、紅、藍」，那新點就是「紅」。

#### 7. K-平均演算法 (K-Means)：

- **概念**：這是一種「非監督式學習」（沒有標準答案）。想像你有一堆混在一起的珠子，K-Means 會自動幫你把顏色或大小相近的珠子分成 K 堆。
- **數學原理 (Friendly Math)**：它在找「**重心 (Centroid)**」。
  1. 先隨便選 K 個點當隊長（重心）。
  2. 每個人（數據點）都加入離自己最近的隊長那組。
  3. 隊長重新計算自己這組的中心位置，移動過去。
  4. 重複步驟 2 和 3，直到隊長不再移動為止。
  - 這就像是大家在操場上集合，最後會自然形成幾個小圈圈，每個圈圈都有一個中心點。

---

## 2.3 第三章：深度學習與神經網路 (Deep Learning & Neural Networks)

### 2.3.1 模仿大腦的運作

深度學習 (Deep Learning) 是機器學習的一種特殊形式，它的靈感來自於人類大腦的運作方式。我們的大腦由數十億個神經元組成，它們相互連接傳遞訊號。深度學習使用「人工神經網路 (Artificial Neural Networks)」模擬這種結構。

- **神經元 (Neuron)** :接收輸入訊號 經過處理(權重加權) 決定是否向下傳遞。
- **層 (Layer)** :神經網路通常有好幾層 輸入層接收數據 隱藏層負責複雜的運算 輸出層給出結果「深度」學習就是指有很多層隱藏層。

**Figure Prompt:** Generate a diagram of a Neural Network. Left side: Input Layer (nodes). Middle: Several Hidden Layers (nodes connected by lines). Right side: Output Layer (nodes). Highlight the connections showing data flowing from left to right. Style: Tech, glowing lines, dark background.

## 2.3.2 深度學習的三大巨頭

### 1. 卷積神經網路 (CNN - Convolutional Neural Networks) :

- **專長：看。**CNN 非常擅長處理影像 它像人類的眼睛一樣 會先辨識邊緣 線條 再組合成形狀 最後辨識出物體。
- **應用：**人臉辨識 醫療影像診斷 自駕車看路標。

### 2. 循環神經網路 (RNN - Recurrent Neural Networks) :

- **專長：記。**RNN 擅長處理有順序的數據 它有「記憶」功能 能記住前面的資訊來幫助理解後面的資訊。
- **應用：**語音辨識(理解句子的前後文) 股票預測(時間序列) 語言翻譯。

### 3. Transformer :

- **專長：專注** 這是目前最強大的架構(如 ChatGPT 背後的技術) 它引入了「自注意力機制 (Self-Attention)」 能同時看到整篇文章 並知道哪些字詞之間有強烈的關聯 而不受距離限制。
- **應用：**大型語言模型 (LLM) 生成式 AI。

---

## 2.4 第四章 :建模與調校 (Modeling & Tuning)

### 2.4.1 打造完美的 AI 模型

建立一個 AI 模型就像是訓練一個運動員 需要經過精心的準備和訓練。

#### 1. 數據準備 (Data Preparation) :

- **垃圾進 垃圾出 (Garbage In, Garbage Out)** :如果你給模型錯誤或雜亂的數據 它學出來的東西也會是一團糟。
- **特徵工程** :把數據轉換成電腦好理解的格式 例如 電腦看不懂「紅色」、「藍色」 我們要把它變成數字編碼 (One-hot Encoding)。

- **標準化**：把不同單位的數據(如身高 180 cm 和體重 70 kg)縮放到同一個範圍，避免模型產生偏差。

## 2. 模型訓練與評估 (Training & Evaluation)：

- **訓練集與測試集**：我們通常把數據分成兩份。80% 用來訓練(像課本)，20% 用來考試(像期末考)，看看模型是不是真的學會了，還是只是死背答案。
- **交叉驗證 (Cross-validation)**：為了更客觀，我們會輪流用不同的數據來考試。

## 3. 評估指標 (Metrics)：

- **準確率 (Accuracy)**：考對了幾題？(最直觀，但不一定最好)。
- **召回率 (Recall)**：該抓出來的壞人，抓出了幾個？(在醫療診斷或詐騙偵測中很重要，我們寧可抓錯，不可放過)。

## 4. 過擬合 (Overfitting) 與 欠擬合 (Underfitting)：

- **過擬合**：書讀太死，模型在訓練集(課本)表現滿分，但遇到新數據(考試)就掛了。解決方法：多做題目(增加數據)，不要讀太細(正規化)。
- **欠擬合**：書沒讀懂，模型連訓練集都學不好。解決方法：換個更聰明的腦袋(更複雜的模型)。

這就是 AI 與機器學習的基礎之旅。希望這些概念能幫助你更好地理解這個正在改變世界的技術！

## 3 第二篇：AI 技術應用與生成式 AI (AI Applications & Generative AI)

歡迎來到 AI 的應用世界！如果說第一篇是學習內功(基礎理論) 那麼這一篇就是學習招式(實際應用) 我們將深入探討目前最熱門的「生成式 AI」,以及它如何與傳統的「鑑別式 AI」相輔相成,改變我們的創造與工作方式。

### 3.1 第五章：鑑別式與生成式 AI (Discriminative vs Generative AI)

#### 3.1.1 兩大 AI 陣營的對決與合作

在 AI 的廣闊領域中,我們可以粗略地將模型分為兩大類,它們就像是人類大腦中的「左腦」與「右腦」,各司其職。

##### 3.1.1.1 1. 鑑別式 AI (Discriminative AI) :嚴謹的判官

這類 AI 就像是一位經驗豐富的鑑識專家或法官,它的主要任務是「分類」和「預測」。

- \* 運作方式：它學習數據之間的邊界,給它一張照片,它會判斷「這是貓還是狗?」;給它一筆交易紀錄,它會判斷「這是不是詐騙?」。
- \* 核心邏輯：它尋找的是  $P(Y|X)$ ,也就是在給定輸入  $X$  的情況下,它是類別  $Y$  的機率是多少。
- \* 應用場景：
  - \* 垃圾郵件過濾：判斷這封信是不是垃圾信。
  - \* 人臉辨識：判斷這個人是不是員工。
  - \* 信用評分：判斷這個人會不會還錢。

##### 3.1.1.2 2. 生成式 AI (Generative AI) :天馬行空的藝術家

這類 AI 就像是一位充滿創意的畫家或作家,它的主要任務是「創造」全新的數據。

- \* 運作方式：它學習數據的分佈規律,然後試著模仿並產生新的樣本,它不是在做選擇題,而是在做申論題或繪畫題。
- \* 核心邏輯：它尋找的是  $P(X, Y)$  或  $P(X)$ ,也就是試著去理解數據  $X$  本身長什麼樣子,然後憑空(或根據提示)生出一個新的  $X'$ 。
- \* 應用場景：
  - \* 寫作：幫你寫一首詩、一篇新聞稿(如 ChatGPT)。
  - \* 繪圖：畫出一隻在太空漫步的貓(如 Midjourney)。
  - \* 作曲：創作一段貝多芬風格的音樂。

**Figure Prompt:** Create a split-screen comparison illustration. Left side (Discriminative AI): A robot judge looking at a basket of mixed fruit and sorting them into “Apples” and “Oranges” bins. Right side (Generative AI): A robot artist looking at a fruit basket and painting a completely new, unique fruit on a canvas. Style: Colorful, isometric 3D.

### 3.1.2 當判官遇見藝術家：整合應用

在實際應用中，這兩者往往不是對立，而是合作的。例如，在訓練一個強大的生成模型（如 GAN）時，我們同時需要一個生成器（藝術家）和一個鑑別器（判官）。藝術家努力畫出逼真的假畫，判官努力抓出假畫，兩者在競爭中共同進步，最後藝術家的畫功達到以假亂真的境界。

---

## 3.2 第六章：生成式 AI 應用與工具 (Generative AI Tools & Applications)

### 3.2.1 No Code / Low Code：AI 民主化運動

過去，要開發 AI 應用程式，你必須是精通 Python 的工程師。但現在，**No Code (無程式碼)** 和 **Low Code (低程式碼)** 平台的興起，讓不懂程式設計的一般人也能輕鬆駕馭 AI。

- **No Code**：完全不需要寫任何程式碼，透過「拖拉放 (Drag-and-Drop)」的方式，像堆積木一樣把功能組裝起來。
  - **優勢**：門檻極低，快速驗證點子。
  - **限制**：客製化程度較低，只能用平台提供的積木。
- **Low Code**：需要寫少量的程式碼來進行客製化，適合有一定技術基礎但想加快開發速度的人。

**熱門工具推薦：**\* **Zapier / Make**：自動化神器，你可以設定「當 Gmail 收到信時，自動叫 ChatGPT 幫我寫回覆草稿，並存到 Google Docs」。\* **Coze / Dify**：專門用來打造 AI 機器人的平台，你可以上傳自己的知識庫 (PDF、網頁)，快速做出一個「公司客服 AI」或「法律諮詢 AI」。

### 3.2.2 駕馭 AI 的關鍵技能：Prompt Engineering (提示工程)

擁有最強的 AI 工具（如 ChatGPT），如果你只會問「你好」，那就像開著法拉利去買菜。**Prompt Engineering** 就是與 AI 溝通的藝術，教你如何下達精準的指令，讓 AI 發揮 100% 的實力。

### 3.2.2.1 三大心法：

1. **Zero-shot (零樣本提示)**：直接問，不給範例。
  - Prompt：「將這句話翻譯成英文：『今天天氣真好』。」
  - 適用：簡單、常見的任務。
2. **Few-shot (少樣本提示)**：給它幾個範例，讓它照樣造句。
  - Prompt：「將下列形容詞轉為顏色：天空 -> 藍色；草地 -> 綠色；太陽 -> ？」
  - 適用：需要特定格式或風格的任務。
3. **Chain of Thought (CoT, 思維鏈)**：叫 AI 把思考過程寫出來，不要直接給答案。
  - Prompt：「這道數學題很難，請一步一步推理給我看，最後再給出答案。」
  - 適用：複雜的邏輯推理、數學問題。

**Figure Prompt:** A visual guide to Prompt Engineering. Three panels. Panel 1 (Zero-shot): User says “Translate”, AI outputs text. Panel 2 (Few-shot): User shows flashcards “A=1, B=2”, then asks “C?”, AI says “3”. Panel 3 (Chain of Thought): User asks complex question, AI shows a thought bubble with gears turning and steps “1..2..3..” before speaking.

### 3.2.3 生成式 AI 的導入與風險

企業在引進 AI 時，不能只是一頭熱，必須經過審慎的評估：1. **需求確認**：我們真的需要 AI 嗎？它能解決什麼痛點？2. **資源盤點**：我們有足夠的數據嗎？算力夠嗎？預算多少？3. **風險管理**：\* **幻覺 (Hallucination)**：AI 會一本正經地胡說八道。\* **資安**：員工會不會把公司機密丟給 AI？\* **版權**：AI 生成的圖片會不會侵權？

---

## 3.3 第七章：進階 AI 技術與部署 (Advanced AI Tech & Deployment)

### 3.3.1 深入 AI 的技術核心

除了基礎模型，還有一些進階技術正在推動 AI 的邊界：

1. **GAN (生成對抗網路)**：如前所述，這是兩個神經網路在打架。
  - 應用：Deepfake 換臉、將黑白照片變彩色、修復模糊照片。
2. **Diffusion Model (擴散模型)**：這是目前最強大的繪圖 AI (如 Stable Diffusion) 的核心。

- **原理**：想像把一滴墨水滴入水中(擴散) 圖案變模糊(加噪聲) 擴散模型就是學習這個過程的「倒帶」從一團雜訊中慢慢還原出清晰的圖像。

3. **LLM (大型語言模型)**：基於 Transformer 架構 閱讀了網路上幾乎所有的文字 它們是通用的語言大師。

### 3.3.2 NLP 與 CV：AI 的聽說讀寫

- **自然語言處理 (NLP)**：讓電腦聽懂人話。
  - **詞袋模型 (Bag-of-Words)**：把句子變成單字的集合 不考慮順序 雖然簡單 但在垃圾郵件分類還是很有用。
  - **Word2Vec**：把單字變成向量(數字列表) 神奇的是 它能捕捉語意關係 例如：國王 - 男人 + 女人 = 女王。
  - **BERT / GPT**：現代 NLP 的霸主 能理解上下文的深層含義。
- **電腦視覺 (CV)**：讓電腦看懂世界。
  - **YOLO (You Only Look Once)**：超快速的物件偵測 它看一眼照片 就能框出裡面有幾個人 幾輛車 幾隻狗 而且速度快到可以處理即時影片。
  - **Segmentation (影像分割)**：比偵測更精細 它能把物體的輪廓精確地描出來 像素級的分類 這在醫療影像(標記腫瘤範圍)和自駕車(區分路面和人行道)非常重要。

**Figure Prompt:** A diagram illustrating “Diffusion Model”. From left to right: A clear image of a cat -> slightly grainy cat -> very noisy static -> pure noise. Then an arrow curving back underneath labeled “Reverse Diffusion (Generation)” showing the noise turning back into a clear cat image.

### 3.3.3 AI 系統部署：從實驗室到真實世界

模型訓練好只是第一步 如何讓它在真實世界穩定運作才是挑戰。\* **MLOps**：就像 DevOps 但是針對機器學習 它管理模型的全生命週期：訓練 版控 部署 監控。\* **Edge AI (邊緣運算)**：把 AI 模型瘦身 塞進手機 攝影機或無人機裡 不用連上雲端也能運作 優點是速度快 隱私好(數據不出門)。\* **Cloud AI (雲端運算)**：使用 Google、AWS 的強大伺服器來跑超大模型 優點是算力無限，但需要網路。

這篇我們探討了 AI 的應用面 從生成式 AI 的創意爆發 到 NLP 與 CV 的感知能力 以及最後如何將這些技術落地部署 掌握這些 你就擁有了改變世界的工具箱！

## 4 第三篇：資料科學與大數據 (Data Science & Big Data)

在 AI 的世界裡，演算法是引擎，而**資料 (Data)** 就是燃料。沒有高品質的燃料，再好的引擎也跑不動。這一篇我們將探討如何開採、提煉並使用這些珍貴的數位石油。

### 4.1 第八章：資料處理基礎 (Data Processing Basics)

#### 4.1.1 資料：AI 的糧食

資料科學 (Data Science) 是一門從數據中挖掘知識的學問。在餵給 AI 吃之前，我們必須先處理好這些食材。

##### 4.1.1.1 1. 資料收集與清洗 (Data Collection & Cleaning)

現實世界中的數據往往是髒亂的。\* **資料收集**：來源可能來自資料庫、網頁爬蟲、感測器 (IoT) 或問卷調查。\* **資料清洗**：這是資料科學家花最多時間的地方 (通常佔 80%)。\* **缺失值處理**：有的欄位是空的怎麼辦？補平均值？補 0？還是直接刪掉這筆資料？\* **異常值處理**：年齡欄位出現 200 歲？這明顯是錯誤，需要修正或剔除。\* **格式統一**：日期格式有 2023/01/01 也有 Jan 1, 2023，必須統一才能運算。

##### 4.1.1.2 2. 資料視覺化 (Data Visualization)：讓數據說話

人類對數字不敏感，但對圖形很敏感。好的視覺化能让你一眼看出數據背後的故事。

- **圓餅圖 (Pie Chart)**：適合看**比例**。例如：市佔率 (A 公司 30%, B 公司 20%...)。
- **長條圖 (Bar Chart)**：適合**比較大小**。例如：各部門的營收比較。
- **折線圖 (Line Chart)**：適合看**趨勢**。例如：股價走勢、氣溫變化。
- **直方圖 (Histogram)**：適合看**分佈**。例如：全校學生的身高分佈 (大部分集中在中間，兩邊人少)。
- **散佈圖 (Scatter Plot)**：適合看**相關性**。例如：唸書時間 vs 考試成績 (通常是正相關，點點往右上跑)。
- **熱力圖 (Heatmap)**：用顏色深淺代表數值。例如：網站點擊熱圖，紅色代表最多人點的地方。



**Figure Prompt:** A dashboard style illustration showing multiple charts. Top left: A colorful Pie Chart. Top right: A rising Line Chart. Bottom left: A Bar Chart comparing 3 items. Bottom right: A Scatter Plot with a trend line. Style: Modern UI dashboard, dark mode.

---

## 4.2 第九章：大數據技術與應用 (Big Data Technologies)

### 4.2.1 當資料大到無法想像：大數據 (Big Data)

當資料量大到一台電腦存不下、算不完時，我們就進入了「大數據」的領域。大數據通常具備 **3V** 特性：1. **Volume (大量)**：資料量極大 (TB, PB 等級)。2. **Velocity (高速)**：資料產生速度極快 (如即時交易、社群媒體貼文)。3. **Variety (多樣)**：資料格式千奇百怪 (文字、圖片、影片、Log 檔)。

### 4.2.2 大數據的統計學基礎

在處理大數據時，我們依然依賴統計學來幫助我們理解全貌。\* **敘述性統計**：用平均數、中位數、標準差來描述這堆數據長什麼樣子。\* **推論性統計**：從大數據中抽樣一小部分來研究，然後推斷整體的狀況 (因為全部分析可能太慢了)。\* **假設檢定**：科學地驗證你的猜想。例如：「改版後的網站真的有提高轉換率嗎？」還是只是運氣好？

### 4.2.3 駕馭大數據的工具

工欲善其事，必先利其器。處理大數據需要特殊的工具：

#### 1. 分散式運算 (Distributed Computing)：

- **Hadoop (MapReduce)**：大數據的祖師爺。它的概念是「分而治之」，把一個大任務切成一千份，分給一千台電腦算，最後再把結果合併起來。
- **Spark**：Hadoop 的進化版。它把資料放在**記憶體 (RAM)** 裡運算，速度比 Hadoop 快 100 倍，現在是主流。

#### 2. NoSQL 資料庫：傳統的 SQL 資料庫 (像 Excel 表格) 在面對超大數據時會變慢。NoSQL 放棄了一些嚴格的規則，換取極致的速度和擴充性。

- **MongoDB**：文件型。存資料像存 JSON 檔案一樣自由，適合存結構不固定的資料。
- **Redis**：鍵值型 (Key-Value)。資料存在記憶體裡，讀寫速度快如閃電，常用來做快取 (Cache)。
- **Cassandra**：寬欄型。Facebook 開發的，寫入速度極快，適合存大量的 Log 或感測器數據。

### 3. 資料流處理 (Stream Processing) :

- **Kafka** :就像一個超大容量的數位水管 ,它可以承接每秒百萬筆的數據洪流(如使用者點擊、交易紀錄) ,然後穩穩地輸送給後端的系統去處理。

#### 4.2.4 大數據與 AI 的強強聯手

大數據是 AI 的最佳拍檔。\* **鑑別式 AI** :需要大數據來訓練出更準確的模型(如 Google 的廣告推薦)。\* **生成式 AI** :需要海量的文本(整個網際網路的資料)來訓練 LLM (如 GPT-4)。

同時 ,我們也要注意**資料隱私** ,在使用大數據時 ,必須遵守法規(如 GDPR) ,做好去識別化 ,保護用戶的隱私權。

**Figure Prompt:** An illustration of the “3Vs of Big Data” . Three icons arranged in a triangle. 1. Volume: A stack of server racks or hard drives. 2. Velocity: A speedometer or a fast-moving train. 3. Variety: A collage of icons representing video, text, images, and audio. Style: Infographic style.

這篇我們學習了如何處理和分析資料 ,以及在大數據時代不可或缺的技术棧 ,掌握了燃料 ,下一篇我們將探討如何安全地駕駛這輛 AI 跑車——AI 的倫理與治理。

## 5 第四篇：AI 倫理、治理與風險 (AI Ethics, Governance & Risks)

能力越強，責任越大。AI 就像一把雙面刃，用得好可以造福人類，用不好則可能帶來災難。在這一篇，我們將探討如何安全、負責任地開發和使用 AI。

### 5.1 第十章：AI 治理與風險 (AI Governance & Risks)

#### 5.1.1 AI 的潛在威脅

AI 並不是完美的，它也會犯錯，甚至被惡意利用。我們必須認識這些風險，才能有效地防範。

##### 1. AI 幻覺 (AI Hallucination)：

- **現象**：AI 有時候會一本正經地胡說八道。例如，問它「林肯總統是哪一年發明燈泡的？」，它可能會編造一個看起來很真實的年份，但事實上林肯根本沒發明燈泡。
- **原因**：生成式 AI 的本質是「機率預測」，它只是在預測下一個字最可能接什麼，而不是在查證事實。
- **對策**：對於關鍵事實（如醫療、法律建議），必須進行人工查核 (Human-in-the-loop)。

##### 2. 偏見 (Bias)：

- **現象**：如果訓練數據本身就有偏見，AI 就會學壞。例如，如果訓練數據中大部分的工程師都是男性，AI 可能會誤以為「女性不適合當工程師」，在篩選履歷時歧視女性。
- **對策**：使用多樣化、平衡的數據集，並在模型上線前進行公平性測試。

##### 3. Deepfake (深偽技術)：

- **現象**：利用 AI 生成極度逼真的假影片或假聲音。這可能被用來製造假新聞、詐騙（假裝親人聲音借錢）或毀壞他人名譽。
- **對策**：開發 Deepfake 偵測工具，並推動數位浮水印技術。

##### 4. 對抗式攻擊 (Adversarial Attacks)：

- **現象**：駭客在圖片上加入人類看不見的微小雜訊，就能騙過 AI。例如，在一張熊貓的照片上加一點點雜訊，AI 竟然會把它誤判成長臂猿。
- **對策**：進行對抗式訓練 (Adversarial Training)，讓模型看過這些攻擊樣本，增強抵抗力。

## 5. 模型漂移 (Model Drift) :

- **現象** : 世界在變 數據也在變。一個半年前訓練好的模型 現在可能已經不準了。例如 消費者的購物習慣改變了 原本的推薦系統就會失效。
- **對策** : 持續監控模型效能 並定期重新訓練 (Retrain)。

**Figure Prompt:** An illustration of “AI Hallucination” . A robot wearing glasses is reading a book and confidently telling a story to a human, but the speech bubble contains a mix of real facts and obvious fantasy elements (like a unicorn). The human looks confused. Style: Cartoonish, humorous.

---

## 5.2 第十一章：AI 倫理 (AI Ethics)

### 5.2.1 AI 應該遵守的道德規範

除了技術上的風險 我們還必須考慮 AI 對社會的影響。AI 倫理就是一套指導原則 確保 AI 的發展符合人類的價值觀。

#### 5.2.1.1 1. 隱私保護 (Privacy)

AI 需要大量數據 這往往涉及個人隱私。\* **資料匿名化 (Data Anonymization)** : 在收集數據時 去除可以識別個人身分的資訊 (如姓名、身分證號) 只保留分析所需的特徵。\* **用戶同意 (Consent)** : 必須明確告知用戶我們會收集什麼資料 用來做什麼 並取得用戶同意。

#### 5.2.1.2 2. 透明度與可解釋性 (Transparency & Explainability)

AI 不應該是一個黑盒子 (Black Box)。\* **問題** : 如果 AI 拒絕了你的貸款申請 你一定想知道「為什麼？」如果銀行說「不知道 是 AI 算的」這顯然無法接受。\* **原則** : 對於影響重大的決策 (如醫療、司法、金融)，AI 的決策過程必須是可解釋的 讓人們理解它是依據什麼邏輯做出判斷。

#### 5.2.1.3 3. 公平性 (Fairness)

AI 應該公平地對待每一個人 不分種族、性別、年齡或宗教。\* **算法公正性** : 開發者必須意識到自己的偏見 並積極消除模型中的歧視。

### 5.2.2 案例分析：AI 倫理的兩難

- **自駕車的電車難題**：如果自駕車煞車失靈，前方有五個路人，轉向會撞死一個路人（或乘客自己），AI 該如何選擇？這是一個沒有標準答案的倫理難題。
- **醫療診斷輔助**：AI 可以幫忙看 X 光片，但如果 AI 誤診了，責任在誰？是醫生？是 AI 開發商？還是醫院？

**Figure Prompt:** A conceptual illustration of “Explainable AI”. On the left, a “Black Box” AI with a question mark, outputting a decision “Rejected”. On the right, a “Glass Box” AI (transparent) showing the internal gears and logic, outputting “Rejected because: Income too low”. Style: Comparative diagram.

AI 治理不僅僅是技術問題，更是法律、社會和哲學問題。只有建立完善的治理框架，我們才能安心地享受 AI 帶來的便利。

## 6 第五篇：AI 程式語言 (AI Programming)

想親手打造自己的 AI 嗎？那你必須學會與電腦溝通的語言。在 AI 領域，這個語言只有一個霸主，那就是 Python。

### 6.1 第十二章：Python 基礎 (Python Basics)

#### 6.1.1 為什麼是 Python？

Python 之所以成為 AI 界的通用語言，原因很簡單：1. **簡單易學**：它的語法非常接近英文，讀起來像在讀文章，而不是在解密碼。2. **生態系豐富**：擁有海量的 AI 函式庫（如 TensorFlow, PyTorch, Scikit-learn），別人已經幫你寫好 90% 的程式碼了，你只需要學會怎麼「呼叫」它們。

#### 6.1.2 Python 語法速成

讓我們從最基礎的語法開始，踏出程式設計的第一步。

##### 6.1.2.1 1. 變數 (Variables)：資料的容器

變數就像是一個貼了標籤的箱子，用來裝資料。  
\* **命名規則**：\* 可以包含字母、數字、底線 `_`。  
\* **不能以數字開頭**：`3_Pig` 是錯的，`Pig_3` 是對的。  
\* **區分大小寫**：`Apple` 和 `apple` 是兩個不同的箱子。  
\* **範例**：

```
name = "Alice"    # 字串 (String)
age = 25           # 整數 (Integer)
height = 1.65      # 浮點數 (Float)
is_student = True  # 布林值 (Boolean)
```

#### 6.1.2.2 2. 輸出 (Print) :讓電腦說話

用 `print()` 函數把結果顯示在螢幕上。\* 基本用法：`print("Hello World")` \* 進階用法：可以用逗號分隔多個東西，預設會用空白隔開。

- 範例：

```
print(5, 10)          # 輸出: 5 10
print(5, 10, sep=',') # 輸出: 5,10 (指定分隔符號為逗號)
```

#### 6.1.2.3 3. 運算子 (Operators) :數學計算

- 基本運算：`+`, `-`, `*`, `/` (除法 結果是小數)。
- 特殊運算：
  - `//` 求商數 (地板除法)：`7 // 2` 等於 3 (只取整數部分)。
  - `%` 取餘數：`7 % 2` 等於 1 (7 除以 2 餘 1)。
  - `**` 次方：`2 ** 3` 等於 8 (2 的 3 次方)。

#### 6.1.2.4 4. 邏輯控制 (Logic Control) :決定程式的走向

- `if...else` (如果...否則)：

```
score = 80
if score >= 60:
    print("及格")
else:
    print("不及格")
```

#### 6.1.2.5 5. 迴圈 (Loops) :重複做一樣的事

- `for` 迴圈：當你知道要跑幾次，或要遍歷一個清單時。

```
for i in range(5): # 跑 5 次 (0 到 4)
    print(i)
```

- `while` 迴圈：當你不知道要跑幾次，只知道停止條件時。

```
count = 0
while count < 5:
    print(count)
    count += 1
else:
    print("跑完了") # while...else: 當迴圈正常結束時執行
```

- **控制指令：**

- **break**：立刻跳出迴圈(不跑了)。
- **continue**：跳過這一次，直接進入下一輪。

#### 6.1.2.6 6. 資料結構 (Data Structures)：收納資料的櫃子

- **List (列表)**：最常用的，什麼都能裝，且有順序。

- `fruits = ["apple", "banana", "cherry"]`
- 可以用 `fruits[0]` 拿到 “apple”。

- **Tuple (元組)**：跟 List 很像，但是不可變 (Immutable) 一旦建立就不能修改。

- `point = (10, 20)`
- 適合用來存座標或設定檔，保護資料不被誤改。

- **Dictionary (字典)**：用「鍵-值對 (Key-Value)」來存資料，像查字典一樣。

- `student = {"name": "Bob", "age": 20}`
- 可以用 `student["name"]` 拿到 “Bob”。

- **Set (集合)**：一堆不重複的資料，沒有順序。

- `numbers = {1, 2, 2, 3}` -> 實際存的是 {1, 2, 3} (自動去重)。
- 常用來做集合運算(交集、聯集)。

**Figure Prompt:** A visual cheat sheet for Python Data Structures. Four quadrants. 1. List: A train with numbered cars [0], [1], [2]. 2. Tuple: A sealed glass box containing items (Immutable). 3. Dictionary: A library card catalog with “Key” labels pointing to “Value” books. 4. Set: A basket of unique fruits (no duplicates allowed). Style: Cute, educational illustrations.

恭喜你！你已經完成了這本書的旅程，從 AI 的基本概念，到機器學習的原理，再到實際的應用與程式實作，這只是起點，AI 的世界浩瀚無垠，期待你繼續探索，用 AI 創造更美好的未來！